QuadraFormer: Unified Query and Resource Forecasting for Database Workloads

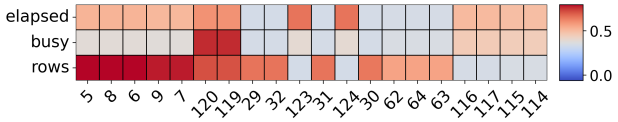
Songwei Han[1], Jiangtao Cui[1], Luming Sun[2], Yingfan Liu[1⋆], Zhangqian Mu[1], Ming Li[2]
[1]*School of Computer Science and Technology, Xidian University*, Xi'an, 710071, China
[2]*Shanghai Yunxi Technology Co., Ltd., Yunxi*, Shanghai, 200120, China
{hansw2114, zqm}@stu.xidian.edu.cn, {cuijt, liuyingfan}@xidian.edu.cn,{sunluming, liming2017}@inspur.com

***Abstract*—Modern database systems rely on workload forecasting to guide performance tuning, resource allocation, and query optimization. However, evolving query patterns and resource demands make accurate workload forecasting increasingly challenging. Most existing methods separately model query and resource behaviors using simple statistics or single-scale forecasting models, thereby ignoring their inherent correlations and leading to inaccurate modeling of real-world workload behaviors. In practice, real-world workloads show that query and resource behaviors are often interdependent and exhibit temporal dynamics. However, most forecasting methods overlook the inherent correlations between query and resource, as well as the temporal patterns across multi-scale resolutions in real workloads. To address these issues, we propose QuadraFormer, an end-to-end forecasting framework that jointly predicts query and resource usage. First, we introduce a unified workload representation that integrates query and resource information into a structured input, enabling joint modeling of workload across variates. Second, we develop an adaptive multi-scale routing mechanism that selects appropriate temporal resolutions to capture periodic patterns and multi-scale variability. Third, we design a novel Quadra-attention mechanism that models both intra- and inter-patch temporal dependencies as well as intra- and inter-dimensional dependencies. Extensive experiments on real-world workloads show that QuadraFormer consistently outperforms competitive baselines, achieving the highest F1 score of 98.38% and accuracy of 86.17%, with F1 improvements of up to 10.28% over the best-performing baseline, while reducing training time by up to 2.16×.**

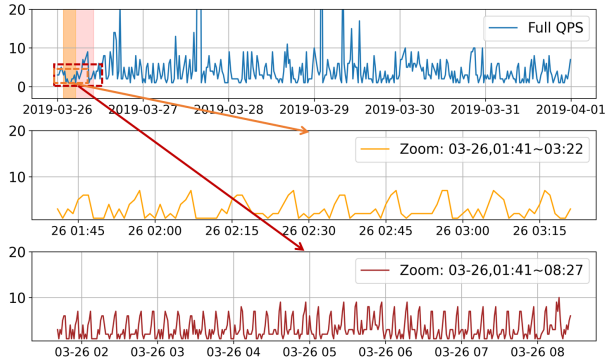
***Index Terms*—Workload Forecasting, Query Prediction, Resource Forecasting, Multi-Scale Timeseries, Transformer**

I. Introduction

Database systems support foundational applications across industries, making accurate workload forecasting essential for effective resource management [1], query optimization, and autonomous tuning tasks [2] such as knob adjustment [3], index recommendation [4], and capacity planning. A workload trace typically comprises two interdependent components: query-level behavior, including SQL template identifiers, associated parameter values, and their execution timestamps; and system-level resource usage, including CPU, memory, and I/O operations. These two aspects often evolve jointly over time. In recent years, production workloads have become increasingly complex, high-volume, and temporally diverse, exhibiting both periodic patterns and multi-scale variability. Such characteristics pose considerable challenges for workload forecasting, demanding greater adaptability and representational capacity.



(a) Correlation heatmap between query parameters (horizontal axis) and resource usage metrics (vertical axis).



(b) Multi-scale QPS traces showing diverse temporal patterns.

Fig. 1: Visualization of workload-query dependencies and multi-scale dynamics on the SDSS workload.

As shown in Fig. 1, real-world workloads often exhibit two notable characteristics: strong correlations between query parameters and system resource usage, and temporal dynamics spanning multiple time scales. Specifically, the heatmap in Fig.1a reveals aligned fluctuations in elapsed time, CPU utilization, and I/O metrics, suggesting consistent workload-query interactions. Meanwhile, the QPS series in Fig. 1b exhibits both short-term patterns and long-range periodic trends, illustrating clear multi-scale temporal dynamics. These observations motivate the design of a unified and temporally adaptive forecasting framework that can capture both cross-dimensional dependencies and multi-scale temporal dynamics.

However, most existing workload forecasting approaches focus on a single variate, modeling either system-level resource usage or query-level semantics in isolation. Resource-centric methods such as QB5000 [5], DBSeer [6], LightPro [7], and TEALED [8] rely on coarse-grained signals and overlook the

⋆Corresponding author
This work was partially supported by the National Natural Science Foundation of China (Grant No. 62372352) and the Special Task Project of the Ministry of Industry and Information Technology of China (No. ZTZB-23-990-024).

2374-8486/25/$31.00 ©2025 IEEE
DOI 10.1109/ICDM65498.2025.00039

influence of query structure and parameter variability. On the query side, models like Sibyl [9] and Q-learning [10] target fine-grained parameter prediction by capturing temporal patterns or query transitions, yet ignore downstream system dynamics. While a few workload-aware frameworks, such as DBAugur [11], have attempted joint modeling through end-to-end pipelines, they typically oversimplify query behavior as aggregate QPS and fail to incorporate parameter semantics or explicit query–resource interactions. Meanwhile, general-purpose Transformer models [12]–[16] advance multivariate time-series forecasting through attention and patch-level modeling, but treat input dimensions independently and lack query-specific priors. Joint modeling remains challenging due to the tight coupling between query parameters and resource usage, and the complex multi-scale temporal patterns found in real workloads. These issues arise from two limitations: (1) the lack of a unified representation that integrates query and resource usage, and (2) insufficient modeling of temporal dynamics and cross-dimensional dependencies.

To address these issues, we propose QuadraFormer, an end-to-end forecasting framework that jointly predicts query parameters and system resource usage. First, we introduce a unified workload representation that integrates query and resource information into a structured input, enabling the model to capture their temporal alignment and semantic dependencies across variates. Second, we develop an adaptive multi-scale routing mechanism that dynamically selects appropriate temporal resolutions based on input characteristics, allowing the model to capture both short-term fluctuations and long-term periodic trends without manual scale tuning. Third, we design a novel Quadra-attention mechanism that extends traditional attention to four dependency types: intra-patch temporal, inter-patch temporal, intra-dimensional, and inter-dimensional, enabling joint reasoning over temporal dynamics and dimensional dependencies in a unified, generalizable manner. Together, these components allow QuadraFormer to effectively bridge the modeling gap in real-world workload forecasting scenarios.

The main contributions of this paper are summarized as follows:

- We propose a unified workload representation that captures both query parameters and system resource usage, enabling joint modeling of two traditionally decoupled variates.
- We enable fine-grained modeling of intra- and inter-patch temporal dynamics, as well as intra- and inter-dimensional dependencies, supported by a unified attention architecture and an adaptive multi-scale routing mechanism.
- We conduct extensive experiments on three real-world workload traces, showing that QuadraFormer achieves superior performance and generalizes well compared with state-of-the-art methods.

The remainder of this paper is structured as follows: Section II reviews related work; Section III details the QuadraFormer methodology; Section IV describes the experimental setup and presents comprehensive experimental results and analyses; and Section V concludes the paper.

II. Related Works

Prior efforts in workload forecasting can be broadly categorized into query-level prediction, resource usage modeling, and general-purpose multivariate time-series forecasting. We briefly review representative methods from each line and discuss their limitations in supporting unified workload modeling.

A. Workload Forecasting

a) Query Trace Forecasting: Research in query trace forecasting primarily predicts future query structures or specific parameter values based on historical workload patterns. Sibyl [9] employs stacked LSTM networks to capture fine-grained parameter variations of SQL query templates. Meduri et al. [10] propose a Q-learning framework modeling query transitions as a Markov decision process to predict the next likely query. Zhou et al. [17] introduce a graph embedding approach representing concurrent queries as dependency graphs to capture structural relationships, improving multi-query prediction accuracy.

While these models capture short-term transitions and query parameters, they generally lack unified workload representations and overlook resource usage patterns. Moreover, existing methods often exhibit limited temporal adaptability, restricting their ability to capture complex temporal structures and long-range dependencies present in real-world workloads.

b) Resource Trace Forecasting: Resource utilization forecasting has evolved from traditional statistical models to hybrid and deep learning frameworks. DBSeer [6] leverages regression over workload templates to model concurrency behavior in OLTP systems. QB5000 [5] employs query template clustering combined with regression trees and LSTM networks, providing per-template QPS forecasting with limited adaptability. DBAugur [11] integrates multi-layer perceptrons (MLPs), temporal convolutional networks (TCNs), and WFGAN within a mixture-of-experts architecture, balancing smooth and periodic workload patterns. LightPro [7] applies Gaussian mixture modeling and matrix factorization for efficient forecasting in DBaaS environments. TEALED [8] utilizes empirical mode decomposition combined with auto-LSTM to achieve long-horizon and frequency-aware workload forecasting. Recent approaches also explore transformer-style architectures and multi-head attention mechanisms to improve modeling capacity for non-stationary workloads [18].

Nonetheless, these methods primarily model aggregate resource metrics such as QPS, CPU, and I/O independently from query parameters, typically treating forecasting as a single-task regression problem. These methods often treat forecasting as single-task regression and overlook structured query–resource dependencies crucial for accurate workload modeling.

B. Multivariate Time-Series Forecasting

Multivariate time-series forecasting has seen rapid advancements driven by deep learning architectures [19]. Classical

models such as LSTM [20] capture sequential dependencies through recurrent structures but often struggle with long-range patterns. Transformer-based models overcome this limitation by leveraging self-attention mechanisms. Informer [12] introduces sparse attention for efficient long-sequence forecasting. FEDformer [13] enhances frequency-domain modeling via seasonal-trend decomposition, while Autoformer [14] leverages autocorrelation to extract periodic signals. PatchTST [21] formulates forecasting as patch-level sequence modeling without positional encoding, improving locality capture. TimesNet [15] uses 2D temporal variation modeling for general-purpose multivariate forecasting. PathFormer [16] and ScaleFormer [22] further adopt multi-scale or fixed-scale attention strategies to enhance temporal granularity modeling.

However, most existing MTSF models treat input dimensions independently and lack domain-specific structural priors, limiting their ability to jointly capture the semantic relationships between query behavior and resource consumption. These are the gaps that QuadraFormer aims to address. Crossformer [23] takes a step forward by introducing hierarchical attention to model cross-variable dependencies, but it remains agnostic to domain semantics such as query execution patterns or resource coupling in database workloads. Our work builds on these insights by incorporating cross-dimensional attention in a unified, query-resource-aware setting.

III. Methodology

This section presents the methodology of QuadraFormer. We begin by formalizing the workload forecasting objective and defining a unified workload modeling formulation. Then, we describe the construction of structured workload traces from query and resource logs. Next, we introduce the QuadraFormer architecture, which integrates adaptive patch routing and quadra-attention mechanisms to model complex workload patterns. Finally, we detail the training and inference procedure under both next-k and next-ΔTsettings.

A. Problem Formulation

- ***Database Workload:***We represent the database workload as a multivariate time series of l historical steps, where each step consists of a SQL query and its corresponding system resource usage. Specifically, we define:

$$\mathbf{x}_t = [q_t; \mathbf{r}_t] \in \mathbb{R}^D, \quad \text{for } t = 1, 2, \ldots, l$$

where q_t denotes the encoded query parameters, and $\mathbf{r}_t$ represents the aligned resource metrics at timestamp t. Each $\mathbf{x}_t$ is a unified input vector after preprocessing and feature engineering.

- ***Next-k Forecasting:*** The full input sequence is denoted by:

$$\mathbf{X} = [\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_l] \in \mathbb{R}^{l \times D}$$

Given the historical input $\mathbf{X}$, the primary forecasting objective is to jointly predict the next k steps of workload behavior, including both query parameters and resource metrics:

$$\hat{\mathbf{X}} = [\hat{\mathbf{x}}_{l+1}, \hat{\mathbf{x}}_{l+2}, \ldots, \hat{\mathbf{x}}_{l+k}] \in \mathbb{R}^{k \times D}$$

where each $\hat{\mathbf{x}}_t = [\hat{q}_t; \hat{\mathbf{r}}_t]$ denotes the predicted query–resource pair at future step t.

- ***Next-ΔT Forecasting:*** As a variant of the above, the next-ΔT forecasting task aims to generate future workload predictions until the cumulative timestamps reach a target horizon ΔT (e.g., 1 hour). This is accomplished by recursively applying next-k forecasting in an autoregressive manner until the time span $[l, l + \Delta T]$ is fully covered.

B. Overall Framework Architecture

As illustrated in Figure 3, QuadraFormer forecasts future workload behavior based on structured historical traces. The input originates from raw query and resource logs, which are parsed and encoded by a pre-processing module to construct a multivariate workload trace containing parameter embeddings and resource metrics.

The forecasting pipeline begins with Reversible Instance Normalization (RevIN) [24]. Then, a multi-scale patching mechanism adaptively selects temporal resolutions based on workload variation. The transformed sequences are passed through a multi-expert attention block to model complex dependencies across both time and feature dimensions. Finally, outputs from different resolutions are fused via a multi-scale aggregator to produce the final prediction.

This modular and hierarchical design allows QuadraFormer to effectively capture multiscale temporal dependencies and cross-dimensional interactions, enabling unified forecasting of query parameters and system resources.

C. Pre-processing: Workload Trace Construction

We transform each workload log into a structured representation that encodes both temporal features and query-specific structure. This section describes the construction of structured workload traces from raw query and resource logs. We extract timestamp features, encode query templates and parameters, and align resource usage metrics to form a unified input sequence.

1) Query Template Representation: Each raw query log entry consists of a timestamp and an SQL statement, denoted as a pair: `(timestamp, SQL)`. We transform this input into a structured representation composed of three parts: fine-grained time features, a query template ID, and corresponding parameter values.

```
SELECT z,
    ra, dec, bestObjID
FROM specObj
WHERE class = $1
AND ra BETWEEN $2 AND $3
AND dec BETWEEN $4 AND $5
AND z BETWEEN $6 AND $7
```

(a) Query template

```
$1 = 'galaxy',
$2 = 130.837,
$3 = 131.037,
$4 = 35.728,
$5 = 35.928,
$6 = 1.0,
$7 = 1.5
```

(b) Query parameters

Fig. 2: Illustration of the query normalization and templating process. The raw SQL is parsed into a template and a corresponding parameter vector.

The preprocessing involves two stages: normalization and templating. The normalization step standardizes SQL formatting by resolving aliases, removing extraneous whitespace, and unifying syntax. The templating step abstracts literal constants using symbolic placeholders, e.g., `$1`, `$2`, and extracts the binding between placeholders and actual values, enabling consistent representation across semantically similar queries. Such template-based abstraction facilitates downstream modeling and is conceptually aligned with prior embedding-based representations such as Query2Vec [25].

For example, the raw query is transformed into:

- Timestamp is decomposed into six numeric features: year, month, day, hour, minute, and second.
- The SQL template is canonicalized and hashed into a categorical ID.
- Query parameters are extracted and passed to a type-aware embedding module.

This structured format captures both fine-grained temporal patterns and structural differences across query templates.

2) Parameter Encoding with Variation Features: Each query template is linked to a set of typed parameters extracted during the templating phase. We categorize parameters by data type and encode each using a dedicated strategy:

- **Numerical**: used in raw form as continuous values such as `int` and `float`.
- **Boolean**: encoded as binary scalars using values like 0 and 1.
- **String**: mapped to categorical indices through feature hashing [26].
- **Temporal**: decomposed into year, month, day, and hour components, each treated as a separate numerical feature.

To enhance temporal sensitivity, we introduce a variation feature for each parameter slot. It is defined as the difference between the current and previous parameter values within the same query template context. This feature captures short-term semantic changes and value transitions in workload patterns.

All encoded features, including type-aware embeddings and variation indicators, are concatenated following the parameter order in each query template. As templates vary in both parameter types and counts, resulting in a variable-length vector aligned to the template's structural definition.

3) Resource Trace Normalization: For each query instance, we extract resource usage metrics from query-level execution profiling. This ensures that all resource features are aligned with individual queries rather than aggregated over time slices, preserving fine-grained workload characteristics.

The collected metrics include, but are not limited to:

- CPU usage: percentage (%)
- Memory consumption: megabytes (MB)
- I/O wait time: milliseconds (ms)
- Disk throughput: megabytes per second (MB/s)
- Execution latency: milliseconds (ms)

Each resource dimension is normalized to the range $[0, 1]$ using min-max scaling based on training set statistics. This normalization is performed independently per dimension. Missing values are imputed with zeros to maintain tensor shape and avoid introducing external noise.

In addition to the raw values, we compute a *variation feature* for each resource dimension, defined as the absolute difference from the previous value. These features enable the model to capture fine-grained resource fluctuations, including transient overloads and recovery dynamics.

The final resource vector concatenates normalized metrics with their variation features. This simple fusion preserves complete temporal dynamics while avoiding complex transformation biases. Combined with timestamp encodings, template identifiers, and parameter embeddings form the unified input at each query step.

D. Forecaster: QuadraFormer Architecture

As illustrated in Figure 3, the Forecaster module adopts a Transformer-based architecture with modular components to model the temporal evolution of database workloads. It takes as input a unified workload trace and applies a learned multi-scale routing mechanism that adaptively selects appropriate temporal resolutions based on workload variation.

The selected sequence segments are divided into overlapping patches at multiple resolutions and passed through a series of Multi-Expert blocks. These blocks jointly model both intra- and inter-patch dependencies, as well as intra- and inter-dimension correlations, enabling fine-grained multi-step forecasting of query parameters and resource usage metrics.

1) RevIN-Based Input Normalization: To improve training stability and mitigate the effects of non-stationary input distributions, we apply RevIN as a lightweight normalization module. For each workload trace in the batch, we normalize the input along the temporal axis by subtracting its mean and dividing by its standard deviation:

$$\tilde{\mathbf{X}} = \frac{\mathbf{X} - \mu}{\sigma} \tag{1}$$

where $\mu, \sigma \in \mathbb{R}^D$ denote the mean and standard deviation of each feature dimension within the current instance. The normalized input $\tilde{\mathbf{X}}$ is fed into the QuadraFormer architecture, producing the normalized prediction $\hat{\tilde{\mathbf{X}}}$. During inference, we apply the inverse transformation to restore the predicted workload trace:

$$\hat{\mathbf{X}} = \hat{\tilde{\mathbf{X}}} \odot \sigma + \mu \tag{2}$$

This reversible normalization ensures that the model learns from stabilized inputs while producing predictions in the original feature space, which is essential for downstream workload management applications.

2) Adaptive Multi-Scale Routing and Aggregation: To capture both short-term variations and long-term periodicities in workloads, we adopt a data-driven multi-scale routing mechanism. Motivated by the sparse routing strategy in PathFormer [16] but differing in its static rule-based scale selection, we incorporate a learnable mechanism that selects relevant temporal resolutions for each input.

We define a candidate set of patch sizes $\mathcal{S} = \{S_1, S_2, \ldots, S_M\}$, each representing a temporal resolution.

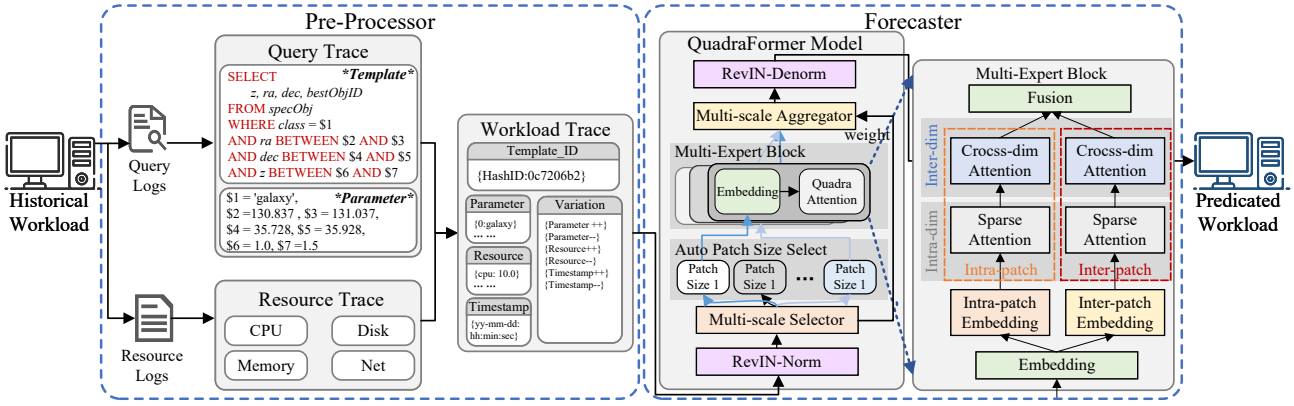


Fig. 3: Unified architecture of QuadraFormer. The model processes structured query–resource traces through normalization, adaptive multi-scale routing, and shared multi-expert blocks that incorporate intra-/inter-patch temporal modeling and intra-/inter-dimensional attention to generate joint multi-step forecasts.

For each input trace $\mathbf{X} \in \mathbb{R}^{l \times D}$, we compute a global summary vector $\mathbf{v} \in \mathbb{R}^{d_s}$, where d_s denotes the dimensionality of the summary embedding. This vector is derived by aggregating variation-aware statistics.

The router generates a soft routing weight vector:

$$\boldsymbol{\alpha} = \mathrm{Softmax}\big(\mathbf{v}W_p + \epsilon \cdot \mathrm{Softplus}(\mathbf{v}W_{\mathrm{n}})\big) \tag{3}$$

where $W_p, W_n \in \mathbb{R}^{s \times M}$ are trainable projection matrices, and $\epsilon \sim \mathcal{N}(0,1)$ introduces stochasticity. The noise encourages exploration during training and prevents premature convergence to suboptimal patch selections.

A Top-K mask is then applied to sparsify $\boldsymbol{\alpha}$, retaining only the K most salient patch sizes. Each selected patch size S_i triggers a parallel attention block with shared weights, performing intra- and inter-patch modeling under its resolution. Let $\hat{\hat{\mathbf{X}}}_i \in \mathbb{R}^{k \times D}$ denote the predicted output from patch size S_i. The final normalized forecast is aggregated as:

$$\hat{\mathbf{X}} = \sum_{i \in \mathcal{K}} \alpha_i \cdot \hat{\hat{\mathbf{X}}}_i \tag{4}$$

where each $\alpha_i \in \mathbb{R}$ is a scalar weight for patch size S_i, and $\hat{\hat{\mathbf{X}}}_i \in \mathbb{R}^{k \times D}$ is the corresponding output.

This instance-adaptive routing–aggregation design allows QuadraFormer to emphasize relevant temporal scales, enhancing forecasting accuracy and generalization across diverse workload patterns. The resulting representations are then passed to the multi-expert block for fine-grained modeling of temporal and cross-dimensional dependencies.

3) Multi-Expert Block with Quadra-Attention: The Multi-Expert Block serves as the core representation module in QuadraFormer, designed to capture both temporal and dimensional dependencies through dual attention branches and residual integration. It consists of two parallel attention branches, Patch-wise and Dimension-wise, that independently model temporal dynamics and cross-dimensional correlations—with their interactions detailed in the Forecaster of Fig. 3. This separation enables efficient learning of both localized and global contextual dependencies.

All attention heads in QuadraFormer adopt sparse dot-product attention [27] with a fixed sliding window. For each query position i, attention is restricted to a local neighborhood of keys within $[i-w, i+w]$, enabling efficient localized computation. This reduces the attention complexity from $O(l^2)$ to $O(lw)$, which is critical for scalability in long-sequence forecasting while preserving locality-sensitive modeling capacity.

Formally, the attention output is defined as:

$$\mathrm{SparseAttn}(Q,K,V)[i] = \mathrm{Softmax}\left(\frac{Q_i K_{[i-w:i+w]}^{\top}}{\sqrt{d}}\right) V_{[i-w:i+w]} \tag{5}$$

where the attention scores outside the window are masked with $-\infty$, resulting in strictly local attention.

a) Patch-wise Attention: This branch captures both short- and long-range temporal dependencies by operating at the patch level. Each input sequence is divided into non-overlapping patches $\{\mathbf{X}^{(1)}, \ldots, \mathbf{X}^{(P)}\}$, where $\mathbf{X}^{(p)} \in \mathbb{R}^{S \times D}$ denotes the p-th patch.

- *Intra-patch attention* models local temporal dynamics within each patch:

$$\mathrm{Attn}_{\mathrm{intra}}^{(p)} = \mathrm{SparseAttn}(Q_{\mathrm{intra}}^{(p)}, K_{\mathrm{intra}}^{(p)}, V_{\mathrm{intra}}^{(p)}) \tag{6}$$

where $Q_{\mathrm{intra}}, K_{\mathrm{intra}}, V_{\mathrm{intra}} \in \mathbb{R}^{S \times d}$ are projections of $\mathbf{X}^{(p)}$ along the temporal axis.

- *Inter-patch attention* captures long-range temporal relations across patches:

$$\mathrm{Attn}_{\mathrm{inter}} = \mathrm{SparseAttn}(Q_{\mathrm{inter}}, K_{\mathrm{inter}}, V_{\mathrm{inter}}) \tag{7}$$

where $Q_{\mathrm{inter}}, K_{\mathrm{inter}}, V_{\mathrm{inter}} \in \mathbb{R}^{P \times d}$ are computed from patch-level pooled summaries.

b) Dimension-wise Attention: While temporal attention captures intra-feature dynamics, the Dimension-wise branch models cross-dimensional interactions within each patch. For each patch index i, we collect all feature vectors:

$$\mathbf{h}_i = [h_{i,1}, h_{i,2}, \ldots, h_{i,D}] \in \mathbb{R}^{D \times d} \tag{8}$$

and apply sparse self-attention along the feature dimension:

$$\text{Attn}^{(i)}_{\text{dim-inter}} = \text{SparseAttn}(Q_i, K_i, V_i) \tag{9}$$

where $Q_i, K_i, V_i \in \mathbb{R}^{D\times d}$ are linear projections of $\mathbf{h}_i$. Here, d denotes the hidden dimension used in attention projections.

This multi-expert block enables QuadraFormer to adaptively focus on salient temporal and cross-dimensional patterns, facilitating unified modeling of query parameters and resource metrics. The fused representation is then projected into multi-step query and resource forecasts under joint supervision.

E. Training Objectives and Inference Procedure

1) Unified Huber Loss: We adopt the Huber loss [28] as a unified optimization objective for both parameter prediction and resource forecasting. For discrete query parameters (e.g., string placeholders), we treat their embeddings as continuous vectors to align with the regression nature of Huber loss, while retaining direct regression for continuous resource metrics. Both targets share this learning objective:

$$\mathcal{L}_\delta(\hat{x}, x) = \begin{cases} \frac{1}{2}(\hat{x} - x)^2, & \text{if } |\hat{x} - x| \le \delta \\ \\ \delta \cdot (|\hat{x} - x| - \frac{1}{2}\delta), & \text{otherwise} \end{cases} \tag{10}$$

The loss is computed element-wise for both query and resource predictions at each forecast step, and aggregated across the forecast horizon to yield the final training objective.

2) Training Procedure: We train the model using a supervised learning setup. At each step, we sample a historical window $\mathbf{X}_{t-\ell+1:t} = [\mathbf{x}_{t-\ell+1}, \ldots, \mathbf{x}_t]$ from the workload trace. The model jointly predicts the next k query parameters and resource vectors:

$$\{\hat{\mathbf{q}}_{t+1}, \ldots, \hat{\mathbf{q}}_{t+k}\}, \quad \{\hat{\mathbf{r}}_{t+1}, \ldots, \hat{\mathbf{r}}_{t+k}\}$$

The total loss is computed as:

$$\mathcal{L}_{\text{Huber}} = \sum_{i=1}^{k} \left(\mathcal{L}_\delta(\hat{\mathbf{q}}_{t+i}, \mathbf{q}_{t+i}) + \mathcal{L}_\delta(\hat{\mathbf{r}}_{t+i}, \mathbf{r}_{t+i})\right) \tag{11}$$

This joint optimization framework allows QuadraFormer to learn temporally aligned representations for both query parameters and resource metrics, forming the foundation for accurate multi-step forecasting.

3) Forecasting Phase: We support two inference modes depending on the application scenario:

- **Next-k Forecasting:** Given a historical window $\mathbf{X}_{t-\ell+1:t}$, the model directly predicts future workload characteristics for the next k queries, producing $\{\hat{\mathbf{q}}_{t+1:t+k}, \hat{\mathbf{r}}_{t+1:t+k}\}$. This setting is used for fixed-horizon multi-step forecasting and standard evaluation.
- **Next-ΔT Forecasting:** In deployment, we aim to forecast the workload trace over a time interval $[t, t + \Delta T]$, where the number of future queries is unknown. The model repeatedly applies next-k forecasting in an auto-regressive manner until the cumulative predicted timestamps cover the target horizon. At each step, the predicted query $\hat{\mathbf{q}}_{t+i}$ is fed back into the model as input to generate the next output. This mode supports practical tasks such as QPS estimation and resource provisioning.

In summary, QuadraFormer combines structured input encoding, adaptive temporal routing, and cross-dimensional attention to jointly forecast workload queries and resource usage in a unified, efficient framework.

IV. Experiments

This section evaluates the forecasting performance of QuadraFormer. We begin by detailing the experimental setup, including datasets, evaluation metrics, and implementation configurations. We then present results under two forecasting settings: fixed-step next-k and interval-based next-ΔT, followed by downstream case studies on QPS forecasting.

A. Experimental Setup

Datasets: We evaluate QuadraFormer on three representative real-world workload traces spanning query-only, resource-only, and joint forecasting scenarios.

- **BusTracker** [29]: A query-only trace from a public transit tracking system. It contains 57 days of SQL query logs from Nov. 29, 2016, to Jan. 25, 2017, totaling approximately 1.1 million queries. The full trace size is about 800MB.

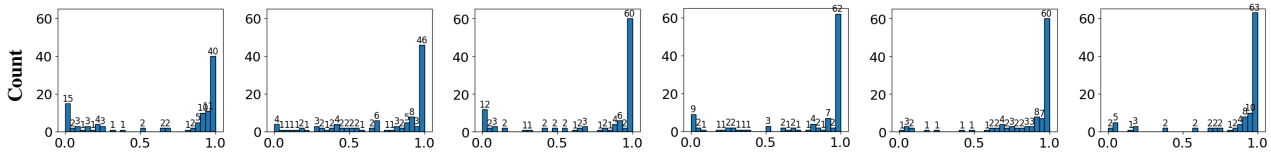
- **Alibaba Cluster Trace** [30]: A resource-only trace from a large-scale production cluster. It includes fine-grained resource metrics, i.e., CPU, memory, disk I/O, and network I/O, sampled every 100ms. We extract a 6-day window from a single representative machine and synthesize pseudo-query sequences from batch task metadata. The raw trace covers 8GB across 4000 machines.

- **SDSS** [31]: A full workload trace from the Sloan Digital Sky Survey (SDSS), containing both SQL query logs and resource usage. We use a 6-day window from Mar. 26, 2019 to Mar. 31, 2019, covering approximately 410,000 queries. The raw trace size is about 200MB.

Each dataset is preprocessed based on its available variates. For SDSS, we extract time-aligned pairs (q_t, r_t), where q_t includes template ID and parameter features, and r_t contains resource metrics. For BusTracker and Alibaba, only the available variates are used. All traces are split chronologically into training and testing sets to preserve temporal continuity.

Baselines: We compare QuadraFormer against a diverse set of baselines spanning statistical methods, deep learning models, and workload-specific architectures. For fair comparison, all baselines are integrated into our unified framework by replacing only the Forecaster module while keeping the data preprocessing and evaluation pipeline consistent.

- **HistoryMean**: A simple baseline that predicts future values as the average of recent observations, assuming short-term stability without modeling temporal trends.
- **QB5000** [5]: A hybrid workload forecaster combining linear regression, kernel regression, and LSTM at the query template level.
- **DBAugur** [11]: An ensemble GAN-based forecaster combining MLPs, TCNs, and WFGAN to model local trends and periodicity.



(a) QB5000 (b) DBAugur (c) Stacked LSTM (d) Transformer (e) PathFormer (f) QuadraFormer

Fig. 4: Distribution of query parameter prediction accuracy across models on SDSS dataset. Horizontal axis: ACC (prediction accuracy); Vertical axis: count of dimensions with corresponding ACC.

- **Stacked_LSTM**: A two-layer LSTM adapted from the Sibyl [9] architecture for query-level workload modeling. As Sibyl does not release source code, we reimplement this baseline based on its reported structure.
- **Transformer [32]**: A vanilla encoder-only Transformer that encodes the input window via self-attention and predicts future steps using a feed-forward decoding head. We adapt its decoder with task-specific heads for mixed query-resource data, differing from its original general design.
- **PathFormer** [16]: A multi-scale patch-based Transformer designed for workload modeling. We follow its official implementation for reproducibility.

These baselines cover a broad range of modeling strategies, allowing us to comprehensively evaluate the strengths and limitations of QuadraFormer under various forecasting scenarios.

Evaluation Metrics: We use distinct metrics for two forecasting tasks: fixed-step next-k forecasting and interval-based next-ΔT forecasting. Each task involves both query parameter prediction and resource usage forecasting, with distinct accuracy criteria for symbolic and numeric targets.

a) Next-k Forecasting: This setting evaluates fixed-horizon forecasting of query trace and resource trace using:

- **ACC**: The proportion of correctly predicted predicate parameters in the query trace.
- **CNT-DIFF**: Relative difference in predicate range coverage, computed for range predicates such as `BETWEEN`, `LIKE`, and `IN`, using the formula $\frac{|\hat{S}-S|}{|S|}$, where $\hat{S}$ and S denote predicted and ground-truth value ranges.
- **MSE**, **MAE**: Regression errors for resource trace. MSE penalizes larger errors more heavily due to squaring, while MAE provides a more robust measure of average deviation.

Parameter matching follows the strategy in Sibyl [9]. For range predicates such as `BETWEEN`, `IN`, or `LIKE`, predictions are considered correct if they overlap with the ground truth. For equality predicates, an exact match is required.

b) Next-ΔT Forecasting: This setting evaluates predicted workloads over a time interval $[t, t + \Delta T]$, using a containment-based F1 score. We construct a bipartite graph between predicted and ground-truth queries, and apply greedy matching to extract one-to-one predicate-containment pairs. Let W and $\hat{W}$ denote the ground-truth and predicted query sets within the interval, and let $\text{match}(W, \hat{W})$ be the number of matched pairs. The F1 score is computed as:

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2 \cdot |\text{match}(W, \hat{W})|}{|W| + |\hat{W}|} \tag{12}$$

All metrics are computed on denormalized outputs. Next-k and next-ΔT results are reported separately.

Implementation Details: All models are implemented in PyTorch [33] and trained on a workstation with an Intel i7-12700K CPU, 40GB RAM, and a single NVIDIA GTX 1080 Ti GPU running Windows 10. We use the AdamW [34] optimizer with a learning rate of 1×10^{-3}, batch size 64, and dropout rate 0.1. Key parameters like sparse attention window size, patch size candidates, routing weights, and Top-K selection are tuned on the validation set to select optimal values. Training runs for up to 20 epochs with early stopping based on validation loss. For the training phase, we use sliding windows, and all resource values are normalized during training and restored for evaluation. We release our implementation and dataset links for reproducibility [1].

B. Main Results

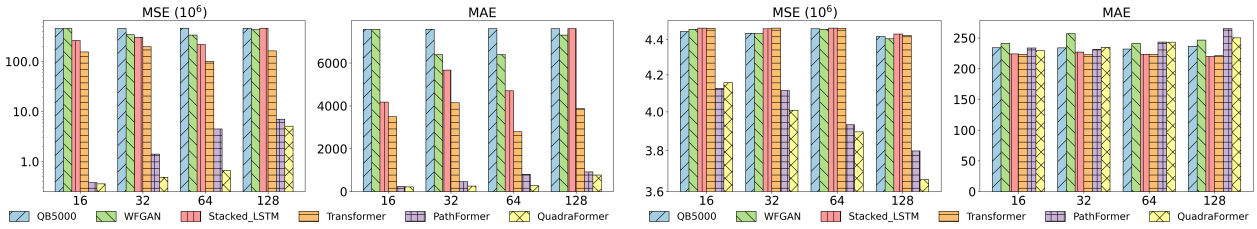
We now present detailed experimental results under two prediction tasks: fixed-step next-k forecasting and interval-based next-ΔT forecasting. Evaluations were conducted on the SDSS, BusTracker, and Alibaba datasets, comparing QuadraFormer against state-of-the-art baseline methods previously described.

1) Next-k Forecasting:

a) Query Parameter Prediction (ACC and CNT-DIFF): Tables I and II present the query-level prediction results. On the SDSS dataset, QuadraFormer consistently achieves the highest accuracy across all k values. Compared to the second-best model, PathFormer, QuadraFormer delivers a relative improvement of over 3.09% in top-1 accuracy at $k = 16$, despite the overall accuracy already nearing saturation. This relative gain is substantial in the context of high-performing baselines, reflecting QuadraFormer's stronger ability to model nuanced query parameters and temporal dependencies. In contrast, methods such as QB5000 and DBAugur perform notably worse; QB5000's static template regression suffers as k increases, while DBAugur remains unstable due to adversarial training dynamics.

CNT-DIFF results exhibit similar trends. QuadraFormer consistently maintains the lowest deviation across all k values

[1] https://github.com/hswhan/QuadraFormer



(a) Alibaba dataset

(b) SDSS dataset

Fig. 5: Comparison of MSE and MAE for different models in next-k forecasting, $k = \{16, 32, 64, 128\}$, on the (a) Alibaba and (b) SDSS datasets.

TABLE I: Accuracy (%) results for next-k forecasting problem using models with fixed window size $K = 32$ across SDSS and BusTracker datasets.

	SDSS				**BusTracker**			
k	16	32	64	128	16	32	64	128
HistoryMean	70.03	67.75	66.95	66.42	55.00	54.65	54.34	54.10
QB5000	67.23	65.68	64.59	65.88	54.43	54.17	54.33	54.30
DBAugur	74.42	65.26	68.74	64.65	55.42	56.74	56.02	55.91
Stacked-LSTM	75.33	76.79	76.02	76.13	57.81	57.17	57.17	58.62
Transformer	77.68	76.58	77.39	76.79	57.85	58.00	58.02	58.87
PathFormer	83.59	81.86	80.33	78.88	59.16	**58.78**	58.56	58.93
QuadraFormer	**86.17**	**84.69**	**81.22**	**79.44**	**59.30**	58.60	**58.97**	**59.01**

and datasets. While PathFormer and Transformer control range variation moderately well, only QuadraFormer shows stable performance even at longer horizons, such as $k = 128$. This indicates that QuadraFormer not only predicts accurate parameters but also preserves predicate count effectively. Notably, some baselines exhibit extremely high CNT-DIFF values on BusTracker; for example, QB5000 reaches 7406 at $k = 128$, indicating severe prediction drift under extended forecasting horizons.

TABLE II: CNT-DIFF metric measuring predicate count deviation in next-k forecasting on SDSS and BusTracker datasets.

	SDSS				**BusTracker**			
k	16	32	64	128	16	32	64	128
HistoryMean	0.65	0.55	0.43	0.41	0.18	0.24	0.27	7406
QB5000	1.33	0.92	1.37	0.82	0.15	0.19	0.17	2583
DBAugur	0.58	0.89	0.97	0.67	0.35	0.15	0.35	2081
Stacked-LSTM	0.50	0.43	0.43	0.33	0.20	0.38	0.39	730
Transformer	0.38	0.31	0.32	0.49	0.19	0.31	0.34	0.36
PathFormer	0.28	0.24	0.36	0.42	0.10	0.11	0.19	0.23
QuadraFormer	**0.20**	**0.21**	**0.28**	**0.35**	**0.01**	**0.01**	**0.01**	**0.01**

Figure 4 visualizes the distribution of query parameter prediction accuracy on SDSS. QuadraFormer yields a sharply peaked distribution near 1.0, indicating consistently accurate predictions. In contrast, methods such as QB5000 and DBAugur exhibit wider variance with distributions skewed towards lower accuracy. PathFormer and Transformer perform better but still show dispersion, while LSTM displays a flatter distribution, reflecting its limited temporal modeling capacity.

b) Resource Usage Forecasting: Figures 5b and 5a provide comparisons of MSE and MAE across models and prediction lengths. On SDSS, QuadraFormer achieves visibly lower MSE and MAE than all baselines. For instance, at $k = 64$ and $k = 128$, it reduces MSE by over 5% compared to PathFormer, and over 10% compared to Transformer or LSTM. Notably, all deep learning models outperform the regression-based QB5000, while DBAugur yields erratic performance due to its generative nature.

On the Alibaba dataset, which contains denser and noisier resource metrics, the advantage of unified modeling becomes more pronounced. QuadraFormer's MSE remains one order of magnitude lower than most baselines, even PathFormer struggles to maintain precision as k increases. This reinforces the necessity of jointly modeling resource metrics and query parameters, an aspect inadequately addressed by traditional methods such as QB5000 or ensemble-based approaches like DBAugur.

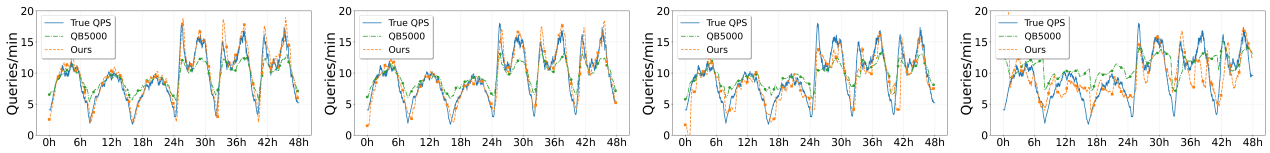
The right-hand MAE plots in both figures also highlight the consistency of QuadraFormer. Even when raw prediction errors vary due to dataset scale, the overall trend shows QuadraFormer exhibiting the flattest MAE curve across increasing k, indicating stable generalization under longer-term forecasting—a key challenge in practical database tuning scenarios.

2) Next-ΔT Forecasting:

a) Interval-based Forecasting: Table III compares models under the next-ΔT forecasting task. On the SDSS dataset, QuadraFormer achieves over 92% F1 at 1-hour resolution and maintains performance above 90% even at 24h. Compared to PathFormer, which peaks at 90.14% but drops sharply at longer intervals, QuadraFormer demonstrates clear robustness over time. Other models like Transformer and LSTM show moderate but flat performance. DBAugur performs well in isolated cases, for example on BusTracker at 1-hour resolution, but collapses under longer windows due to mode collapse during generation.

TABLE III: F_1 (%) for next-ΔT forecasting problem using models with fixed window size $K = 32$.

	SDSS				**BusTracker**			
ΔT	1h	6h	12h	24h	1h	6h	12h	24h
HistoryMean	18.75	48.78	43.75	45.58	28.13	27.44	27.33	29.40
QB5000	14.06	14.11	18.13	20.25	29.17	26.18	25.93	26.12
DBAugur	5.71	34.15	27.22	25.31	30.00	30.36	29.99	29.80
Stacked-LSTM	42.18	41.31	35.34	36.76	31.64	26.93	25.90	28.45
Transformer	43.18	40.52	35.34	37.00	30.27	26.39	26.96	26.97
PathFormer	87.22	90.14	83.78	87.45	30.20	34.44	38.67	**36.80**
QuadraFormer	**92.54**	**98.38**	**92.39**	**95.53**	**35.56**	**40.50**	**39.01**	35.26



(a) $15-MinutesHorizon$ (b) $30-MinutesHorizon$ (c) $1-HourHorizon$ (d) $2-HourHorizon$

Fig. 6: QPS prediction results under different temporal horizons, comparing true values (blue) with predictions of QB5000 (green) and our method (orange) over 48 hours.

On BusTracker, overall F1 scores are lower across all models. This is primarily due to its query-only variate and the dominance of equality predicates, which require exact match under our evaluation protocol. Compared to range predicates, equality predicates are inherently harder to forecast, leading to universally lower accuracy. Despite this challenge, QuadraFormer achieves the best results and demonstrates a smoother F1 degradation curve over longer intervals. In contrast, QB5000 and DBAugur exhibit fluctuating performance, likely due to overfitting to periodic patterns without deeper semantic understanding. These results support our hypothesis that joint modeling of queries and resources, combined with multi-scale adaptation, enhances temporal generalization—especially when anticipating behavior over extended and uncertain windows.

b) QPS Prediction: We evaluate the performance of QuadraFormer in a downstream QPS prediction task over a 48-hour workload trace. The goal is to forecast aggregate query rates across varying temporal horizons, from short-term intervals such as 15 minutes to longer ones up to 2 hours. This task assesses the model's ability to synthesize high-fidelity query volume patterns across time.

Figure 6 compares true QPS in blue, QB5000 in green, and our predictions in orange across four temporal horizons. At fine-grained intervals, our method better captures QPS fluctuations, especially around sharp transitions, than QB5000. As the prediction window expands to 1 or 2 hours, QuadraFormer still preserves the overall workload shape, periodicity, and peak timing, demonstrating strong robustness under increased forecasting uncertainty.

These results highlight QuadraFormer's strength in capturing temporal workload characteristics with high precision. Its ability to generalize across varying horizons makes it especially suitable for practical scenarios such as workload replay, system provisioning, and predictive tuning, where both short-term responsiveness and long-term planning are required.

c) Ablation Study: To further understand the contribution of individual components, we conduct an ablation study on the SDSS dataset. Table IV presents the ablation results on the SDSS dataset with $k = 32$, illustrating the impact of several key components in QuadraFormer: cross-dimension attention, multi-scale routing, sparse attention, and the inclusion of resource traces in the unified representation. The complete QuadraFormer model delivers the best performance across all metrics, achieving 84.69% parameter accuracy, a minimal count deviation of 0.21, and the lowest resource forecasting error with an MAE of 262.19. Notably, when cross-dimension attention is removed, the model suffers a visible drop in ACC, confirming the importance of modeling cross-dimensional interactions for query understanding.

TABLE IV: Ablation results on SDSS with prediction horizon $k = 32$. Each row disables a key component in QuadraFormer to evaluate its impact on performance.

Variant	ACC (%) ↑	CNT-DIFF ↓	MAE ↓	Time/Epoch
QuadraFormer (Full)	**84.69**	**0.21**	**262.19**	671.15s
w/o Cross-dim Attn	83.53	0.26	268.17	429.49s
w/o Multi-scale	81.97	0.31	267.37	**253.29s**
w/o Spare Attention	83.42	0.33	271.42	792.01s
w/o Resource Trace	83.90	0.29	-	675.09s

Removing multi-scale routing results in the most pronounced performance drop, with ACC decreasing to 81.97%, highlighting the importance of adaptive temporal granularity in modeling complex workload patterns. Additionally, replacing sparse attention with standard dense attention slightly reduces MAE but increases training time by over 18%, demonstrating that sparse attention brings clear efficiency benefits with minimal trade-off in accuracy. Finally, removing the resource trace input, which breaks the unified query–resource representation, also degrades parameter prediction quality, increasing CNT-DIFF to 0.29. This validates the core design principle of QuadraFormer: joint modeling of queries and system behaviors enables deeper semantic understanding and more accurate workload forecasting.

C. QuadraFormer vs. PathFormer

Finally, we analyze the computational efficiency by comparing training times of QuadraFormer and PathFormer, as summarized in Table V. QuadraFormer consistently reduces training durations across various datasets at $k = 128$, achieving speedups of 1.77× on SDSS and 2.16× on BusTracker.

This efficiency gap can be attributed to the architectural differences between the two models. PathFormer employs deep stacked layers with full attention across all tokens, which leads to quadratic computational complexity with respect to input length. As the input size grows, either due to larger datasets or longer prediction windows, this design incurs substantial training overhead. In contrast, QuadraFormer leverages sparse attention and adaptive multi-scale routing, which dynamically selects relevant patches and minimizes unnecessary computation. This results in significantly better scalability and faster training without sacrificing accuracy. In summary, the presented results clearly highlight QuadraFormer's superior

TABLE V: Training time per Epoch (s) of PathFormer and QuadraFormer, and speedup over PathFormer, for next-k forecasting.

Dataset	k	**PathFormer**	**QuadraFormer**	**Speedup**
SDSS	16	350.89	249.16	1.41×
	32	984.65	671.15	1.47×
	64	1826.52	1520.32	1.20×
	128	3913.35	2201.11	**1.77**×
BusTracker	16	185.55	155.04	1.20×
	32	304.16	246.50	1.23×
	64	555.51	445.00	1.25×
	128	1917.65	887.97	**2.16**×
Alibaba	16	31.51	15.89	**1.98**×
	32	45.56	25.19	1.81×
	64	65.22	47.72	1.37×
	128	100.66	61.79	1.63×

predictive accuracy, robustness, and computational efficiency, confirming its practical utility in realistic database forecasting scenarios.

V. Conclusion

In this paper, we introduce QuadraFormer, a unified forecasting framework jointly modeling query parameters and system resource metrics in database workloads. Leveraging unified representation and multi-scale Quadra-attention, it captures intricate temporal and cross-dimensional dependencies across workload features. Extensive experiments on diverse real-world traces verify QuadraFormer consistently outperforms strong baselines in both next-k and next-ΔT forecasting tasks. Moreover, it exhibits clear computational advantages, suitable for deployment in automated database systems. Note that QuadraFormer has limited robustness in extreme scenarios (e.g., sudden QPS surges) and weak correlation workloads; future work will address these issues and adapt to diverse database logs like MySQL.

References

[1] X. Lin, H. Wang, Z. Feng, J. Li, and H. Gao, "P-store: A predictive elastic resource management system for cloud-based databases," *Proc. VLDB Endow.*, vol. 13, no. 12, pp. 2992–2995, 2020.

[2] A. Pavlo, C. Curino, and S. Zdonik, "Self-tuning databases: A decade of progress," in *Proc. VLDB Endow.*, vol. 10, no. 12, 2017, pp. 1961–1962.

[3] D. V. Aken, A. Pavlo, G. J. Gordon, and J. M. Patel, "Automatic database management system tuning through large-scale machine learning," in *Proc. ACM SIGMOD*. ACM, 2017, pp. 1009–1024.

[4] B. Mozafari, V. Raman, F. Nargesian, and J. M. Patel, "Lookahead indexing: Adaptive index selection using predictive modeling," in *Proceedings of the 2021 ACM SIGMOD International Conference on Management of Data*. ACM, 2021, pp. 351–364.

[5] L. Ma, D. V. Aken, A. Hefny, G. Mezerhane, A. Pavlo, and G. J. Gordon, "Query-based workload forecasting for self-driving database management systems," in *Proc. ACM SIGMOD*. ACM, 2018, pp. 631–645.

[6] B. Mozafari, C. Curino, A. Jindal, and S. Madden, "Performance and resource modeling in highly-concurrent oltp workloads," in *Proc. ACM SIGMOD*. ACM, 2013, pp. 301–312.

[7] X. Huang, S. Cao, X. Gao, and G. Chen, "Lightpro: Lightweight probabilistic workload prediction framework for database-as-a-service," in *Proc. IEEE ICWS*. IEEE, 2022, pp. 203–212.

[8] C. Yuan, L. Zhao, Y. Chen, B. Liu, and J. Hu, "Tealed: A multi-step resource usage prediction framework with adaptive empirical mode decomposition and auto-lstm," in *Proc. DASFAA*, 2022, pp. 158–173.

[9] H. Huang, T. Siddiqui, R. Alotaibi, C. Curino, J. Leeka, A. Jindal, J. Zhao, J. Camacho-Rodríguez, and Y. Tian, "Sibyl: Forecasting time-evolving query workloads," *Proceedings of the ACM on Management of Data*, vol. 2, no. 1, pp. 1–27, 2024.

[10] V. V. Meduri, K. Chowdhury, and M. Sarwat, "Evaluation of machine learning algorithms in predicting the next sql query from the future," *ACM TODS*, vol. 46, no. 1, pp. 1–46, 2021.

[11] Y. Gao, X. Huang, X. Zhou, X. Gao, G. Li, and G. Chen, "Dbaugur: An adversarial-based trend forecasting system for diversified workloads," in *Proc. IEEE ICDE*. IEEE, 2023, pp. 27–39.

[12] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, and H. Xiong, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI*, vol. 35, no. 12. AAAI Press, 2021, pp. 11 106–11 115.

[13] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "Fedformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 162. PMLR, 2022, pp. 27 268–27 286.

[14] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *NeurIPS*, vol. 34, 2021, pp. 22 419–22 430.

[15] H. Wu, J. Xu, J. Xia, J. Ji, J. Wang, and M. Long, "Timesnet: Temporal 2d-variation modeling for general time series analysis," in *Proc. ICLR*, 2023, presented at ICLR 2023. Available at OpenReview.

[16] P. Chen, Y. Zhang, Y. Cheng, Y. Shu, Y. Wang, Q. Wen, B. Yang, and C. Guo, "Pathformer: Multi-scale transformers with adaptive pathways for time series forecasting," in *Proc. ICLR*, 2024.

[17] X. Zhou, H. Wang, C. Xu, K. Lan, J. Feng, G. Chen, and J. Wang, "Query performance prediction for concurrent queries using graph embedding," *Proc. VLDB Endow.*, vol. 13, no. 9, pp. 1416–1428, 2020.

[18] e. a. Huang, "Workload forecasting using multi-head attention and convolution for probabilistic inference," *IEEE TKDE*, 2022.

[19] Q. Wen, T. Zhou, C. Zhang, W. Chen, Z. Ma, J. Yan, and L. Sun, "Transformers in time series: A survey," in *Proc. IJCAI*. International Joint Conferences on Artificial Intelligence Organization, 2023, pp. 6778–6786.

[20] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[21] Y. Nie, N. H. Nguyen, N. Nguyen, and P. Reiter, "Patchtst: Transformer-based patch modeling for multivariate time series forecasting," in *Proc. ICLR*, 2023.

[22] A. Shabani, P. Malhotra, K. Lerman, and A. Anandkumar, "Scaleformer: Iterative multi-scale refining transformers for time series forecasting," in *NeurIPS*, vol. 35, 2022, pp. 10 138–10 151.

[23] Y. Zhang and J. Yan, "Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting," in *Proc. ICLR*, 2023.

[24] T. Kim, J. Kim, Y. Tae, C. Park, J.-H. Choi, and J. Choo, "Reversible instance normalization for accurate time-series forecasting against distribution shift," in *Proc. ICLR*, 2021.

[25] Y. Nandwani, C. Binnig, M. T. Özsu, and T. Rabl, "Learning representations of sql queries for database workload analytics," in *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data*. ACM, 2019, pp. 1775–1789.

[26] K. Weinberger, A. Dasgupta, J. Langford, A. Smola, and J. Attenberg, "Feature hashing for large scale multitask learning," in *Proc. ICML*. ACM, 2009, pp. 1113–1120.

[27] I. Beltagy, M. E. Peters, and A. Cohan, "Longformer: The long-document transformer," *arXiv preprint arXiv:2004.05150*, 2020.

[28] P. J. Huber, "Robust estimation of a location parameter," in *Breakthroughs in Statistics: Methodology and Distribution*, ser. Springer Series in Statistics, S. Kotz and N. L. Johnson, Eds. Springer, 1992, pp. 492–518.

[29] "Bustracker project," https://github.com/linmagit/QueryBot5000.

[30] "Alibaba cluster data," https://github.com/alibaba/clusterdata.

[31] "Skyserver sql traffic log," https://skyserver.sdss.org/log/en/traffic/.

[32] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *NeurIPS*, vol. 30, 2017.

[33] A. Paszke, S. Gross, F. Massa, A. Lerer, and et al., "Pytorch: An imperative style, high-performance deep learning library," in *NeurIPS*, vol. 32. Curran Associates, Inc., 2019, pp. 8024–8035.

[34] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019, presented at ICLR 2019. Available at OpenReview.